



MANIPAL

ACADEMY *of* HIGHER EDUCATION

(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES
(A Constituent unit of MAHE, Manipal)

CRYPTOLOGY LAB – Assignment 5

Reg. Number	Name	Branch
261100690024	K P NIDHI	CyberSecurity



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

Classic Cryptography Ciphers in Python

Shift Cipher · Caesar Cipher · ROT13 · Atbash Cipher · Affine Cipher

1. Introduction

This assignment implements five classic ciphers in Python: the Shift Cipher, Caesar Cipher, ROT13, Atbash Cipher, and Affine Cipher. Each one has separate encrypt and decrypt functions, keeps letter case as it is, skips over spaces/digits/punctuation, and checks its inputs before running. The code was run in a Jupyter notebook; the screenshots below show each cell along with its output.

2. Shift Cipher

Every letter is moved forward by a fixed number of places (the key). Formula: $E(x) = (x + \text{key}) \bmod 26$, and decryption just uses the negative key.

```
def shift_encrypt(text, key):
    if not isinstance(text, str):
        raise ValueError("Text must be a string")
    if not isinstance(key, int):
        raise ValueError("Key must be an integer")

    result = ""
    for ch in text:
        if ch.isalpha():
            start = ord('A') if ch.isupper() else ord('a')
            result += chr((ord(ch) - start + key) % 26 + start)
        else:
            result += ch
    return result

def shift_decrypt(text, key):
    return shift_encrypt(text, -key)

# test
msg = "Hello, World!"
enc = shift_encrypt(msg, 4)
print("Encrypted:", enc)
print("Decrypted:", shift_decrypt(enc, 4))

Encrypted: Lipps, Asvph!
Decrypted: Hello, World!
```

Figure 1: Shift Cipher - code and output

3. Caesar Cipher

Same as the shift cipher, just with the key fixed at 3 by default.

```
def caesar_encrypt(text, key=3):
    return shift_encrypt(text, key)

def caesar_decrypt(text, key=3):
    return shift_encrypt(text, -key)

# test
enc = caesar_encrypt(msg)
print("Encrypted:", enc)
print("Decrypted:", caesar_decrypt(enc))

Encrypted: Khoor, Zruog!
Decrypted: Hello, World!
```

Figure 2: Caesar Cipher - code and output

4. ROT13

Shift cipher with key 13. Since $13 + 13 = 26$, running it twice gives back the original text, so one function handles both encryption and decryption.

```
def rot13(text):
    return shift_encrypt(text, 13)

# test (running it twice gives back the original)
enc = rot13(msg)
print("Encrypted:", enc)
print("Decrypted:", rot13(enc))

Encrypted: Uryyb, Jbeyq!
Decrypted: Hello, World!
```

Figure 3: ROT13 - code and output

5. Atbash Cipher

No key needed - each letter is swapped with its mirror position in the alphabet (A<->Z, B<->Y, and so on). Also self-inverse.

```
def atbash(text):
    if not isinstance(text, str):
        raise ValueError("Text must be a string")

    result = ""
    for ch in text:
        if ch.isalpha():
            start = ord('A') if ch.isupper() else ord('a')
            result += chr(start + (25 - (ord(ch) - start)))
        else:
            result += ch
    return result

# test (also self-inverse)
enc = atbash(msg)
print("Encrypted:", enc)
print("Decrypted:", atbash(enc))

Encrypted: Svool, Dliow!
Decrypted: Hello, World!
```

Figure 4: Atbash Cipher - code and output

6. Affine Cipher

Uses two keys, a and b . Formula: $E(x) = (a*x + b) \bmod 26$. For decryption to work, ' a ' must be coprime with 26 ($\gcd(a, 26) = 1$), so we find the modular inverse of a to reverse the encryption.

```

from math import gcd

def affine_encrypt(text, a, b):
    if not isinstance(text, str):
        raise ValueError("Text must be a string")
    if gcd(a, 26) != 1:
        raise ValueError("Key 'a' must be coprime with 26")

    result = ""
    for ch in text:
        if ch.isalpha():
            start = ord('A') if ch.isupper() else ord('a')
            x = ord(ch) - start
            result += chr((a * x + b) % 26 + start)
        else:
            result += ch
    return result

def affine_decrypt(text, a, b):
    if gcd(a, 26) != 1:
        raise ValueError("Key 'a' must be coprime with 26")

    a_inv = None
    for i in range(1, 26):
        if (a * i) % 26 == 1:
            a_inv = i
            break

```

Figure 5: Affine Cipher - encrypt/decrypt functions

```

    result = ""
    for ch in text:
        if ch.isalpha():
            start = ord('A') if ch.isupper() else ord('a')
            y = ord(ch) - start
            result += chr((a_inv * (y - b)) % 26 + start)
        else:
            result += ch
    return result

# test
enc = affine_encrypt(msg, 5, 8)
print("Encrypted:", enc)
print("Decrypted:", affine_decrypt(enc, 5, 8))

# validation check
try:
    affine_encrypt(msg, 4, 8) # gcd(4,26) != 1
except ValueError as e:
    print("Error caught:", e)

Encrypted: Rc1la, Oaplx!
Decrypted: Hello, World!
Error caught: Key 'a' must be coprime with 26

```

Figure 6: Affine Cipher - test run and validation check

7. Validation Summary

The code checks that the text is a string and the keys are integers before encrypting, and for the Affine cipher it additionally checks that $\text{gcd}(a, 26) = 1$ (a must be coprime with 26) so that a modular inverse exists for decryption. Anything that fails these checks raises a `ValueError` with a clear message, as shown in the last screenshot where $a = 4$ is correctly rejected.